

CPSY 1291 · ASSIGNMENT 1 CALL SHEET

# Signatures and idioms

Fall 2026 · everything here was taught in Recitations 1-2 · keep it next to the notebook

## Shapes, axis=, keepdims

```
X.shape # (120, 4096) stimuli x features
X.mean(axis=0).shape # (4096,) the named axis disappears
X.mean(axis=1, keepdims=True).shape # (120, 1)
X - X.mean(axis=1) # ValueError: (120,4096) (120,)
```

Compare shapes from the right: each pair equal, or one is 1, or you get the error above. `keepdims=True` keeps a size-1 axis so subtraction lines up; `v[:, None]` is the same idea by hand.

## Boolean masks

```
mask = (taxon == 2) # one boolean per stimulus
X[mask] # just those rows
mask.sum(); mask.mean() # how many; what FRACTION (a rate)
np.isin(cat, [0, 2]) # membership in a set of labels
```

Element-wise logic on arrays: `&` and `·` | or `· ~` not — always with parentheses: `(taxon == 2) | (taxon == 5)`. Writing and/or between arrays raises “truth value ... is ambiguous”.

## The upper triangle, and reordering a matrix

```
iu = np.triu_indices(120, k=1) # a PAIR of index arrays
D[iu] # 7,140 values: every pair once
order = np.argsort(taxon)
D[np.ix_(order, order)] # rows AND columns reordered
D[order, order] # NOT that: 120 diagonal entries
```

`iu` walks the upper triangle in a *fixed pair order*, so `D1[iu]` and `D2[iu]` describe the same pair at every position — the whole reason two such vectors can be correlated. `np.ix_` builds the open grid; `D[order, order]` silently applies the paired rule instead.

## Fancy indexing and argsort

```
D[a, c] > D[a, b] + D[b, c] + 1e-9 # one boolean per triple
np.argsort(v)[:5] # indices of the 5 SMALLEST
np.argsort(v)[::-1][:5] # indices of the 5 LARGEST
```

Two index arrays pull one element *per pair* — 200,000 triples in one line, no loop (compute the distance matrix *once*, then index into it). `argsort` returns *indices*, not values — and a stimulus is always maximally similar to itself, so exclude it from its own neighbor list.

## Comparing two RDMS

```
rho = spearmanr(D1[iu], D2[iu]).statistic
```

`.statistic` is where the number lives (`.pvalue` exists, but the pairs are not independent — do not lean on it). Spearman = Pearson on *ranks*: only the ordering of dissimilarities is meaningful. Never feed the whole matrix: that counts every pair twice and adds  $n$  zeros.

## Condensed vs square

```
d = pdist(X, metric='euclidean') # condensed (n*(n-1)/2,)
D = squareform(d) # square (n, n), and back
```

`imshow` and MDS want *square*; linkage wants *condensed*: `linkage(squareform(D_human), checks=False, method='average')`.

## Fitting a curve, and the straight line

```
popt, _ = curve_fit(f_exp, x, y, p0=[10, 1], maxfev=40000)
y_hat = f_exp(x, *popt) # * unpacks the parameters back in
coef = np.polyfit(x, y, 1) # slope and intercept
y_hat = np.polyval(coef, x) # the line at every x
```

`p0`: start near the right scale — similarities run 0–10, so  $a \approx 10$ . `maxfev`: raise it, or hard fits die with “Optimal parameters not found”. Wrap in `try/except RuntimeError` where a fit is *allowed* to fail (2h says so — that failure is a finding, not a bug).

## Goodness of fit

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

1 = perfect · 0 = no better than predicting the mean · negative = worse than that (a real outcome, not a bug). It is a *ratio*: two  $R^2$  values are comparable only against the same target — binning first raises it with the curve unchanged.

## The scikit-learn pattern

```
est = PCA(n_components=50)
Y = est.fit_transform(X) # items x features in
est.explained_variance_ratio_ # exists only after .fit
cum = np.cumsum(est.explained_variance_ratio_)
np.argmax(cum >= 0.85) # first True: components to 85%
```

Settings go in the constructor, data goes to `.fit`, learned results grow a *trailing underscore* (not a typo). MDS takes a dissimilarity matrix: `metric='precomputed'`. Random methods differ between runs unless you set `random_state` — reproducible is not the same as meaningful.

## Per-group anything (groupby without groupby)

```
means = np.stack([fc6[obj == k].mean(axis=0)
                  for k in range(51)]) # (51, 4096)
```

Four moves: `mask` → those rows → collapse `axis=0` → `np.stack`. Every “for each object / category / cluster” sentence is this template with a different word in step 3.

## NaNs — pick one, and say which

```
np.isnan(A).sum() # count them before trusting anything
IT = it_raw[:, ~np.isnan(it_raw).any(axis=0)] # DROP
IT = np.where(np.isnan(it_raw), # IMPUTE
             np.nanmean(it_raw, axis=0), it_raw)
```

Both are defensible; the notebook only demands you say *which*. Ordering rule: check for zero-variance units *before* z-scoring — dividing by a std of 0 makes NaNs, and one NaN poisons everything downstream. Related clip: `np.sqrt(np.maximum(D2, 0))` — floating point leaves tiny negatives where the true value is 0.

## silhouette\_score — what goes in

```
silhouette_score(proj_fc6[:, :2], (cat == c).astype(int))
```

First argument: the 2-D *coordinates* in the PC1–PC2 plane, not the RDM — it computes its own distances. Second: a binary labeling, this category against everything else. Near 1 = sits apart · near 0 = on a boundary · negative = closer to another group than its own. No relation to 3fs *silhouette masks* — the name is a coincidence.